

第 5 章 零代码编程

在科研工作中，研究人员常常需要处理大量表格数据、整理非结构化文本、完成统计分析与图表绘制，甚至开发课题组内部使用的小型工具。过去，这些任务往往依赖编程基础、统计软件操作和较长时间的调试过程；而在生成式人工智能和编程智能体的支持下，研究人员可以用自然语言描述需求，让 AI 辅助生成代码、运行分析、开发原型系统并保存结果文件。本章将围绕“零代码编程”展开，介绍如何从自然语言提出任务，逐步完成科研小工具开发、智能数据处理、自动化统计分析、学术图表生成，以及智能体驱动的软件原型构建与迭代测试。

5.1 从“写代码”到“自然语言编程”

长期以来，编程能力一直是许多科研人员开展数据处理、模型构建和工具开发时面临的重要门槛。随着大语言模型和编程智能体的发展，科研编程正在出现一种新的交互方式：研究人员可以用自然语言描述任务目标、输入数据、处理逻辑和输出要求，再由 AI 辅助生成代码、运行程序并解释结果。这种方式通常被称为“自然语言编程”。它并不意味着研究人员完全不需要理解计算过程，而是将工作重心从手动记忆语法和逐行编写代码，转向明确需求、拆解任务、检查结果和迭代修改。本节将围绕自然语言编程展开，说明科研人员如何借助 AI 完成从需求表达、代码生成到零代码系统原型开发的基本过程。

5.1.1 编程思维的转变

可以把大模型粗略理解为一种“自然语言到代码”的转换助手，能够理解人类用日常语言描述的任务意图，并将其转化为机器可执行的代码。随着代码执行引擎和编程智能体的发展，编程界面不再局限于传统的代码编辑器，也可以是一个对话框、一个数据分析沙箱，甚至是一个能够读取文件、运行程序、发现报错并自动修复的智能体工作区。换言之，科研人员不一定要亲手敲出每一行代码，但必须学会用清晰、可验证的方式描述自己真正想让程序完成什么。

这种变化标志着编程范式正在从“语法驱动”转向“意图驱动”。编程范式的转变从“语法驱动”到“意图驱动”如图 5-1 所示。在传统流程中，研究人员首先需要学习 Python、R 或 MATLAB 等编程语言的语法规则，然后逐行编写代码、运行程序、排查报错，并不断修改底层实现细节。而在大模型辅助的新范式下，任务重心明显前移：研究人员首先需要思考问题

本身，明确任务目标、输入数据、输出结果、异常情况和验收标准；随后再通过自然语言将这些要求交给大模型或编程智能体，由其生成代码、执行程序并反馈结果。此时，科研人员的角色不再只是“代码书写者”，而更接近“系统设计者”“任务监督者”和“结果审查者”。



图 5-1 编程范式的转变从“语法驱动”到“意图驱动”

因此，不再亲手编写底层代码，并不等同于不再需要计算思维。相反，人工智能时代的计算思维要求更高层次的抽象能力和工程意识。它不再主要表现为记住某个函数的具体写法，而是表现为能否把一个模糊的科研需求拆解成若干清晰的操作步骤；能否识别数据中的规律、噪声和异常；能否把专业问题转化为程序可以执行的逻辑；以及能否判断系统输出是否真实、合理、可复现。简单地说，过去的编程能力更多体现为“我会不会写代码”，现在的编程能力则更多体现为“我能不能把问题定义清楚，并检查机器是否真正完成了任务”。

以批量提取不规则文献数据为例。假设一份包含数千条文献记录的电子表格中，“文本”列的数据格式并不统一，例如有的记录写作“出版于自然杂志，2023 年，第 15 卷”，有的记录写作“2022 年科学期刊论文”，还有的记录可能缺少年份信息。当前的科研任务是从这些文本中单独提取年份，并生成一个新的“年份”列。

在传统方式下，研究人员往往需要学习正则表达式的编写规则，思考如何匹配连续四位数字，如何处理空值、错误格式和特殊符号。如果数据格式稍有变化，原有代码就可能失效。而在具备现代计算思维的新方式下，研究人员首先要做的不是立刻写代码，而是完成任务定义。例如可以向大模型输入如下指令：

“请读取指定的表格文件，提取‘文本’列中代表年份的 4 位连续数字，并将其存入新列‘年份’。如果某一行没有发现年份，则在该行填入‘暂无’。如果同一行出现多个 4 位数字，请优先提取位于‘年’字前后的数字。处理完成后，请输出新的表格文件，并统计成功

提取、未提取和疑似异常的记录数量。请同时随机展示 10 条处理前后的样例，供我人工检查。”

这段指令之所以优于简单地说“帮我提取年份”，是因为它不仅说明了目标，还明确了输入、输出、异常处理方式和结果验收方式。大模型可以据此生成并执行相应代码，但科研人员仍然需要检查输出是否符合研究目的。例如，系统是否把“第 15 卷”误判为年份？是否遗漏了中文表述中的年份？是否把参考文献编号当成出版年份？这些问题无法完全交给模型自行判断，必须由研究人员结合学科知识进行复核。

由此可见，人工智能辅助编程并不是让研究人员放弃思考，而是让思考的位置发生了改变。传统编程要求研究人员把大量精力放在语法记忆、函数调用和报错排查上；自然语言编程则要求研究人员把更多精力放在任务规划、逻辑约束、边界测试和结果解释上。工作的驱动方式已经从“如何写出代码”转变为“如何定义问题、如何约束机器、如何验证结果”。

在这一新范式下，好的自然语言指令不仅是一句话命令，更是一份简洁的任务说明书。它正常需要包含五类信息：第一，任务目标，即希望程序最终解决什么问题；第二，输入数据，即程序需要读取哪些文件、字段或文本；第三，输出形式，即最终应生成表格、图表、报告还是网页工具；第四，异常情况，即遇到缺失值、错误格式、重复记录或不合常理的数据时如何处理；第五，验收标准，即如何判断程序已经正确完成任务。掌握这五类信息的组织方式，比单纯背诵某种编程语言的语法更重要。

这种角色转变为科研工作流带来了新的可能。研究人员可以把琐碎的代码实现交给大模型，把自己的主要精力重新集中在科学假设、实验设计、数据解释和理论建构上。但与此同时，研究人员也必须承担新的责任：不能盲目信任人工智能生成的代码和结果，而要学会通过样例检查、边界测试、重复运行和人工抽查来确认系统是否可靠。只有当“清晰表达需求”和“严格验证结果”同时成立时，自然语言编程才能真正成为科研工作的有效工具，而不是制造错误和幻觉的自动化机器。

5.1.2 零代码开发软件系统

随着大语言模型代码生成能力和编程智能体能力的提升，“零代码开发”的含义正在发生变化。过去，人们提到零代码开发，往往想到的是在专门平台上拖拽按钮、表格和页面组件；而在生成式人工智能时代，零代码开发更多是指研究人员用自然语言描述自己的需求，由大模型自动完成界面设计、功能逻辑、代码生成、运行调试和初步修改。从实现难度看，零代

码开发的软件系统大致可以分为两类：一类是大模型已经非常熟悉的“已知系统”，另一类是带有学科规则和个性化需求的“自定义系统”。

所谓已知系统，是指互联网上已经存在大量成熟实现、功能逻辑高度标准化的软件工具，例如计算器、倒计时器、贪吃蛇游戏、单词测验器、简单问卷页面等。这类系统的特点是目标明确、交互规则成熟、边界较清晰。大模型在预训练过程中已经接触过大量类似代码，因此研究人员通常不需要详细规定每一个底层实现步骤，只需要交代总体目标和必要的功能要求即可。

以开发经典的“贪吃蛇”游戏为例。对于这类已知系统，初学者常犯的错误是把提示词写得过于冗长，例如详细规定方向键如何监听、蛇身如何用坐标表示、食物如何随机生成、碰撞如何判断等。实际上，大模型通常已经掌握这类标准应用的基本结构。研究人员只需要采用较简洁的“意图交付”方式即可：

“请开发一个可以在浏览器中直接运行的贪吃蛇游戏，在页面上方实时显示当前得分，并提供重新开始按钮。”

将这段极简提示词输入后，模型会自动调取其内在的常识库，输出一段集成了前端布局与底层逻辑的代码。生成的贪吃蛇游戏系统界面（DeepSeek 开发）如图 5-2 所示。该系统不仅还原了常规的视觉元素，还处理了底层的按键防抖、状态重置以及画布刷新等复杂逻辑。大家如果想进一步丰富该游戏的功能，比如变成双人模式的，都可以通过自然语言进行沟通。

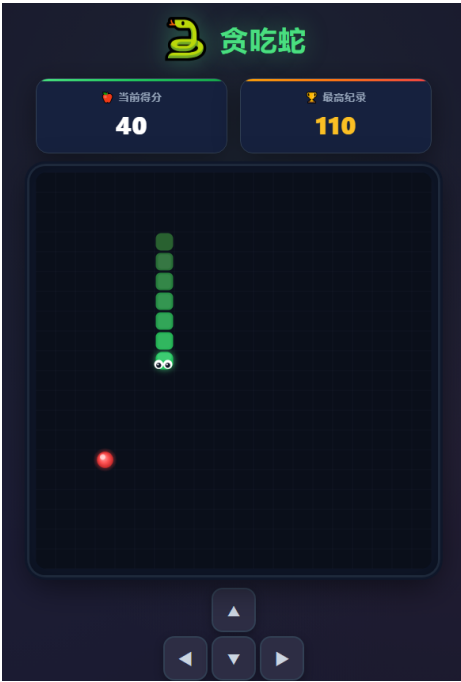


图 5-2 贪吃蛇游戏系统界面展示（DeepSeek 开发）

将这类提示词输入大模型后，系统通常可以生成包含前端布局、游戏逻辑和交互控制的

网页代码。研究人员只需将代码保存为.html 文件，双击后即可在浏览器中运行。通过这个例子可以看出，对于规则成熟、边界清晰的通用系统，零代码开发能够快速完成从想法到原型的转化。

但是，科研工作中真正有价值的工具往往并不是完全通用的，而是带有特定规则、特定数据格式和特定学科判断标准的自定义系统。对于这类系统，大模型虽然知道大致如何开发网页或程序，却并不知道研究人员心中的具体规则。如果提示词过于模糊，模型可能只生成一个外观相似但逻辑不准确的页面，甚至生成无法真正使用的半成品。因此，开发自定义系统时，研究人员必须把需求表达得更加准确，包括目标、输入、输出、约束等。

以开发“个人健康状态评估系统”为例。虽然 BMI 计算本身是常见任务，但如果希望它成为一个具有特定判定标准、动态视觉反馈和交互界面的工具，就必须明确告诉大模型具体规则。自定义健康评估系统指令示例：

“请开发一个个人健康状态评估系统，用于计算并判定用户的肥胖程度。界面需提供三个输入项：性别（下拉菜单）、身高（厘米）、体重（公斤）。点击‘开始评估’按钮后，在下方计算并显示其 BMI 数值。特别规则：必须严格按照特定标准输出结论并动态改变字体颜色：BMI<18.5 显示‘偏瘦’（蓝色）；18.5≤BMI<24 显示‘正常’（绿色）；24≤BMI<28 显示‘超重’（橙色）；BMI≥28 显示‘肥胖’（红色，并附加警告图标）。”

模型接收到这组结构化指令后，便会严格按照划定的边界，生成专门针对该规则的底层逻辑与前端渲染代码。通过 DeepSeek 开发的个人健康状态评估系统界面展示如图 5-3 所示。它不仅完成了身高的单位换算与平方除法运算，更重要的是，它精准实现了你所规定的多重条件判断与动态颜色交互输出。

与简单地说“帮我做一个 BMI 计算器”相比，这段提示词的优势在于，它不仅说明了要做什么，还明确了系统如何接收数据、如何输出结果、采用什么判定标准。大模型据此生成的系统更容易符合研究人员的真实预期。

在零代码开发中，生成代码只是中间步骤，而不是最终目标。系统能够打开，并不代表系统已经正确；页面看起来美观，也不代表底层逻辑可靠。研究人员必须对结果进行验收。例如，对于健康评估系统，除了输入普通样例外，还应测试边界情况：BMI 正好等于 18.5、24 或 28 时分类是否正确；身高为空时是否会报错；体重为负数时是否会被拒绝；输入字母时是否给出友好提示。通过这些边界测试，研究人员才能判断系统是否真正可用。

健康状态评估
BMI 身体质量指数计算器

性别
男性

身高 (厘米) 177 体重 (公斤) 80

开始评估

您的 BMI 指数
25.5
kg/m²
超重

偏瘦 <18.5 正常 18.5-23.9 超重 24-27.9 肥胖 ≥28

图 5-3 个人健康状态评估系统界面展示（DeepSeek 开发）

5.1.3 建立人机协作的开发的边界感

通过自然语言指令快速开发贪吃蛇游戏或自定义系统，极易让初学者产生一种错觉：似乎只要把想法告诉人工智能，它就可以毫无障碍地开发任何软件。然而，在真实的软件工程与科研工具开发中，人工智能的代码生成能力仍然存在清晰的技术边界。尤其是当任务涉及外部数据、复杂交互、多文件项目、服务器部署、用户权限、数据库管理或长期维护时，单次提示词往往很难直接生成稳定可用的完整系统。因此，研究人员在使用人工智能辅助开发时，必须建立一种基本的开发边界感：在提出一个软件想法时，能够初步判断它是适合快速生成的简单任务，还是需要分阶段开发、反复测试和持续迭代的复杂任务。

简单任务通常具有高度的共性，这些共性构成了当前人工智能辅助编程的“舒适区”。例如标准计算器、单位换算器、倒计时器、贪吃蛇游戏、选择题测验器等工具，往往目标明确、功能单一、输入输出清楚，业务逻辑能够在本地形成闭环。更重要的是，这类任务通常不需要连接外部服务器或数据库，也不需要用户登录、文件长期存储、多人协同或复杂权限管理。它们常被设计为“即开即用、用完即走”的轻量化工具。对于这类需求，人工智能往往能够通过一次或少数几次提示词，生成可以直接在浏览器或本地环境中运行的单文件代码。

与此相对，脱离上述特征的需求则会进入人工智能辅助开发的“困难区”。以语言学习软

件为例，让模型生成一个“法语单词背诵测验器”通常属于简单任务，因为它的逻辑主要是本地词库的随机抽取、答案判断和得分统计。但如果要求模型生成一个“带有实时语音纠正反馈的法语陪练系统”，或者一个“能够通过互联网进行多人联机对战的语言学习游戏”，任务的性质就发生了明显变化。这类系统涉及语音识别、发音评估、网络通信、后端服务、数据库存储、用户身份验证和低延迟交互等多个模块，远远超出了单段网页代码所能承载的范围。人工智能在面对这类模糊而庞大的指令时，往往只能生成看似完整、实际上难以运行的半成品框架。

不过，随着 Claude Code、Cursor、OpenCode 等编程智能体的发展，复杂任务并不一定意味着“完全不能做”。更准确地说，复杂任务通常不能依靠单次提示词直接完成，而需要被拆解为若干可实现、可测试、可迭代的小模块。例如，一个“课题组本地文献问答系统”听起来很复杂，但如果将其拆分为 PDF 上传、文本解析、向量索引、问题检索、答案生成、引用显示和错误处理等模块，并为每个模块设定明确的验收标准，编程智能体就可以逐步生成项目文件、运行程序、读取报错、修改代码并推进系统成型。因此，人工智能辅助开发的边界并不是简单的“能做”或“不能做”，而是要看任务是否能够被清晰拆解，并且每一步是否能够被可靠验证。

人工智能辅助开发任务的边界评估图如图 5-4 所示，可以从“逻辑复杂度”和“外部依赖性”两个维度构建判断框架。横轴表示软件自身逻辑的复杂程度：越靠左，说明任务功能越单一、规则越简单；越靠右，说明任务涉及多步骤、多状态或多模块协同。纵轴表示系统对外部资源的依赖程度：越靠下，说明系统主要在本地运行，不依赖外部网络、数据库或服务；越靠上，说明系统需要调用 API、连接数据库、访问网络服务或进行服务器部署。

图 5-4 将人工智能辅助开发任务划分为四个典型区域。左下角是“AI 舒适区”，这类任务逻辑简单、外部依赖少，例如计算器、单位换算器、贪吃蛇游戏和简单测验器。它们最适合初学者快速入门，通常可以通过普通大模型一次生成原型代码。左上角是“依赖区”，这类任务功能本身不一定复杂，但需要外部系统支持，例如调用 API 的小工具、联网查询工具和邮件提醒工具。开发这类系统时，研究人员不仅要写清功能需求，还要注意密钥、接口权限和运行环境配置。右下角是“可拆解区”，这类任务逻辑较复杂，但外部依赖较少，例如数据分析报告、统计绘图工具和算法实验脚本。它们不一定适合一句话直接生成完整系统，但非常适合通过结构化提示词、测试样例和编程智能体逐步开发。右上角是“高风险工程区”，这类任务既有较高逻辑复杂度，又依赖较多外部系统，例如文献问答系统、多人协作平台、实时语音陪练和数据库管理系统。对于这类任务，不能期待人工智能一次性生成稳定可用的完整

系统，而应采用编程智能体开发、人工审查和多轮测试相结合的方式谨慎推进。

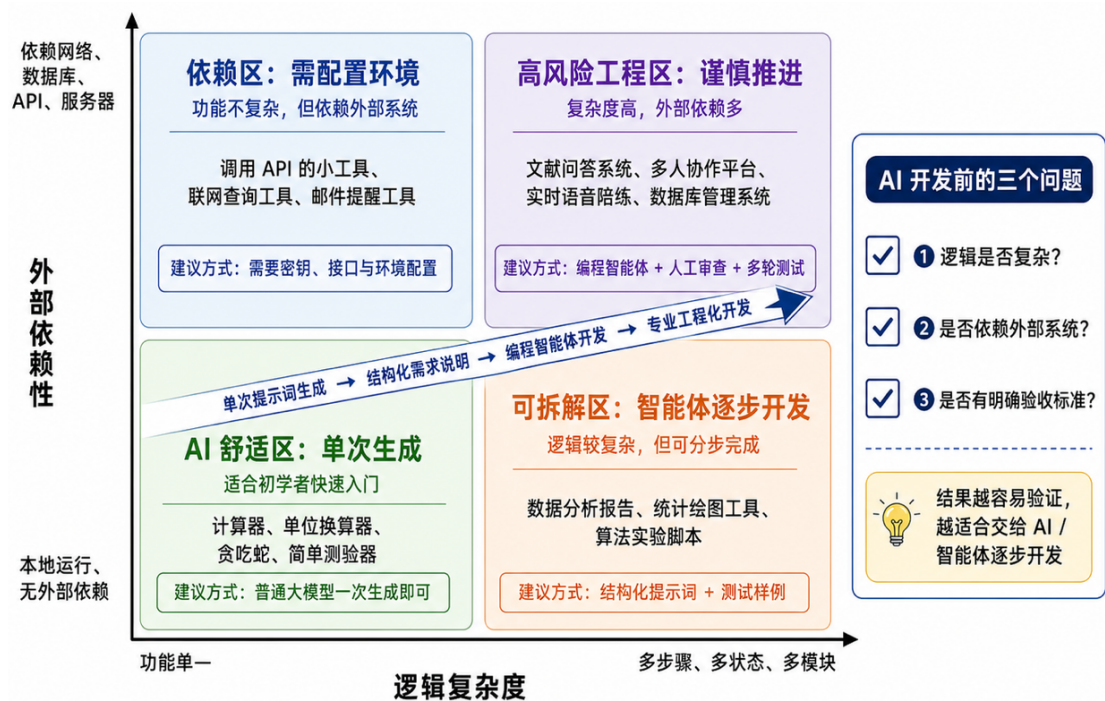


图 5-4 人工智能辅助开发任务的边界评估图

图 5-4 中从左下角指向右上角的斜向箭头，表示人工智能辅助开发方式的升级路径：从“单次提示词生成”，到“结构化需求说明”，再到“编程智能体开发”，最终进入“专业工程化开发”。这一路径提醒大家，任务越复杂，越不能把人工智能当作一次性代码生成器，而应把它作为可以协助拆解任务、生成模块、运行测试和迭代修复的开发伙伴。

除了逻辑复杂度和外部依赖性之外，研究人员还必须特别关注第三个关键因素：结果可验证性。新图右侧列出了人工智能开发前需要思考的三个问题：第一，逻辑是否复杂？第二，是否依赖外部系统？第三，是否有明确验收标准？其中，验收标准尤其重要。一个任务即使比较复杂，只要输入输出明确、测试样例充分、错误可以复现，就可以交给人工智能或编程智能体逐步完成。相反，如果任务判断标准模糊、结果无法核验、成功与失败难以界定，那么即便表面上看起来简单，也很容易产生错误或幻觉。

因此，在构思一个软件系统时，研究人员不应只问“人工智能能不能做”，而应进一步追问：“这个任务能否被拆解？每一步能否测试？最终结果能否验证？”如果答案是肯定的，那么这个任务就更适合采用人工智能辅助开发；如果答案是否定的，则应先回到需求整理阶段，补充输入输出、约束条件和验收样例，再让模型或编程智能体介入。

5.2 软件与工具开发的新范式

随着人工智能辅助编程门槛的显著降低,研究人员不仅能够快速构建轻量化的日常辅助工具,也正在进入一种新的科研软件与智能分析系统开发范式。过去,构建一个具备智能处理能力的科研工具,例如文献情感倾向分类器、访谈材料编码器或实验数据异常检测器,往往高度依赖传统机器学习流程。这通常意味着人工标注数据、编写训练代码、反复调参验证以及部署应用系统,对于多数非计算机专业研究新生而言具有较高门槛。生成式人工智能改变了这一局面。一方面,研究人员可以通过提示词直接调用大模型的语义理解能力,让模型完成文本分类、信息提取、态度判断和开放式问卷初筛等任务;另一方面,随着 Claude Code、Cursor 等编程智能体的发展,研究人员还可以将数据、目标、评价指标和验收要求交给智能体,由其自动编写机器学习代码、运行实验、输出指标、绘制图表并生成实验报告。

5.2.1 智能数据处理工具开发的三种范式

在生成式人工智能普及之前,若要在科研中开发一个专属的智能数据处理工具,通常需要依赖传统机器学习范式。以社会学、心理学、教育学或公共管理研究中常见的“受访者评论情感与态度分类器”为例,研究人员如果希望将数千条评论自动标记为“支持”“反对”或“中立”,往往需要先收集语料并进行人工标注,再完成数据清洗、特征工程、模型训练、参数调优和效果评估等一系列步骤。对于非计算机专业研究新生而言,这一流程不仅技术门槛较高,而且开发周期较长,通常需要较多标注数据和较系统的机器学习知识支撑。

传统机器学习范式并没有失去价值。对于正式的算法研究、高可靠预测任务或需要系统比较不同模型性能的研究,它仍然是严谨而必要的方法。但在许多科研场景中,研究人员的目标并不是提出一种全新的算法,而是希望快速构建一个能够辅助文本分类、信息提取、数据初筛或实验分析的智能工具。生成式人工智能的出现,使这类工具的开发路径变得更加多样。研究人员既可以通过提示词直接调用大模型的语义理解能力,也可以借助编程智能体自动搭建机器学习实验流程。

因此,当前智能数据处理工具的开发大致可以分为三种范式:传统机器学习范式、提示词直接推理范式和智能体算法实验范式。三者并不是简单的替代关系,而是适用于不同任务目标、数据条件和严谨性要求的工具选择。智能数据处理工具开发的三种范式对比如图 5-5 所示。

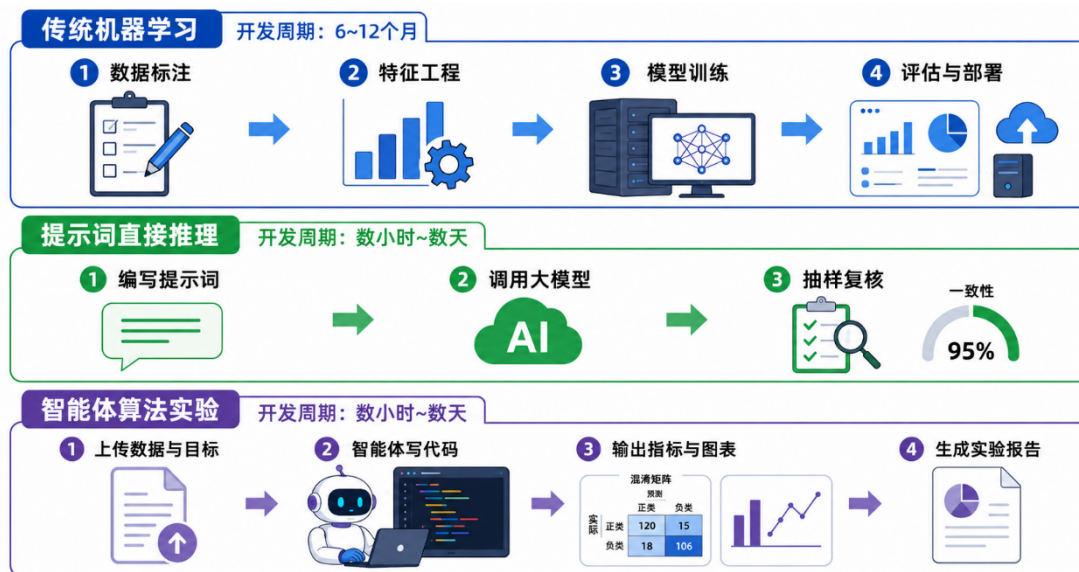


图 5-5 智能数据处理工具开发的三种范式对比

第一种是传统机器学习范式。在这种范式下，研究人员需要准备带标签的数据集，选择合适的算法模型，编写训练和评估代码，并通过验证集或测试集判断模型效果。它的优势是过程相对可控，适合正式模型训练、算法比较和高要求预测任务；不足之处是对数据量、编程能力和机器学习知识要求较高，开发周期通常较长。

第二种是提示词直接推理范式。在这种范式下，研究人员不再从零训练一个新模型，而是通过自然语言提示词，让已经预训练好的大模型直接完成分类、提取或判断。例如，对于受访者评论分类任务，研究人员可以写清分类标准，并提供少量示例，让大模型直接判断评论属于“支持”“反对”还是“中立”。这种方式特别适合文本分类、情感倾向识别、开放式问卷初步编码、访谈材料初筛和文献信息提取等任务。它的优势是启动快、代码量少、使用门槛低；但输出结果可能受到提示词措辞、上下文顺序和模型版本变化的影响，因此必须配合抽样复核、一致性检查和错误分析。

第三种是智能体算法实验范式。它与提示词直接推理不同：提示词直接推理是让大模型直接给出判断结果，而智能体算法实验是将数据文件、任务目标、评价指标和验收要求交给编程智能体，由其自动撰写代码、运行实验、输出指标、绘制图表并生成实验报告。例如，研究人员可以把一个带标签的评论数据集交给 Claude Code、Cursor 等编程智能体，要求其建立文本分类基线模型，输出 Accuracy、Macro-F1 和混淆矩阵，并将实验过程整理成报告。这种方式适合机器学习基线实验、算法对比、数据分析管线和可复现实验报告生成。它的优势是能够把“想法”转化为可运行的实验流程；但研究人员仍然必须检查数据读取、训练测试划分、评价指标、运行日志和代码复现性，不能直接把智能体输出视为最终可信结论。

从这三种范式可以看出，生成式人工智能并不是简单地“取代传统机器学习”，而是在不同层面降低了科研工具开发的门槛。对于只需要快速判断文本含义或提取信息的任务，可以优先考虑提示词直接推理；对于需要训练模型、比较指标和输出图表的任务，可以借助智能体算法实验；而对于严肃的模型创新、高风险预测或需要严格性能验证的任务，仍然应遵循传统机器学习的规范流程。

因此，研究人员在选择开发方式时，不应只问“哪种方法最快”，而应根据任务性质进行判断：如果任务标准明确、结果易于人工复核，可以使用提示词直接推理快速形成初步结果；如果任务需要可运行代码、评价指标和实验记录，可以使用编程智能体搭建实验管线；如果研究目标本身是模型性能优化或算法创新，则仍需要系统的数据标注、模型训练和严格验证。理解这三种范式的差异，是后续学习提示词推理、智能体算法实验、非结构化数据处理和结果校验的基础。

5.2.2 提示词直接推理

提示词直接推理，是指研究人员不再从零训练一个新的机器学习模型，而是通过自然语言指令，直接调用已经预训练好的大语言模型完成分类、提取、判断或归纳任务。这种方式特别适合那些依赖语义理解、但不一定需要复杂模型训练的科研场景，例如开放式问卷初步编码、访谈材料态度归类、文献摘要信息提取等。

与传统机器学习相比，提示词直接推理最大的特点是启动速度快。研究人员不需要首先构建大规模标注数据集，也不需要编写完整的训练代码，只需要把任务目标、分类标准和输入文本表达清楚，大模型就可以立即给出结果。对于非计算机专业研究新生而言，这种方法能够显著降低智能数据处理工具的使用门槛，使其快速完成数据初筛和初步分析。

以社会学或公共管理研究中常见的“受访者评论情感与态度分类”为例。假设研究人员需要判断公众对某项新政策的态度，将评论分为“支持”“反对”或“中立”三类。最简单的方式是使用零样本提示，即不提供任何示例，只用自然语言说明任务和输出要求：

“作为社会学数据分析助手，请判断以下受访者评论的情感态度。你只能输出三个类别之一：支持、反对或中立。

评论文本：这项政策极大地减轻了我的经济负担，流程也简化了，非常感谢！”

在这种提示下，大模型通常能够根据评论中的积极表达，直接输出“支持”。零样本提示的优点是简单、快捷，适合分类标准比较直观的任务。但在真实科研数据中，许多文本并不

如此清晰。例如，有些评论可能既肯定政策目标，又批评执行过程；有些评论可能使用反讽、委婉表达或双重否定；还有些评论可能没有明确态度，只是在描述事实。对于这类边界模糊的样本，仅靠零样本提示往往不够稳定。

为了让大模型更好地对齐研究人员的分类标准，可以使用少样本提示。少样本提示是在任务说明中加入 3 到 5 个具有代表性的“输入—输出”示例，让大模型通过上下文学习理解研究人员的判断尺度。例如：

“作为社会学数据分析助手，请分类受访者评论的情感态度。你只能输出：支持、反对或中立。

【分类标准】

支持：整体上认可政策、制度或措施，并表达正面评价。

反对：整体上否定政策、制度或措施，或强调其造成负面影响。

中立：没有明显支持或反对态度，主要是事实陈述、观望或信息不足。

【参考示例】

示例 1：评论：虽然设备采购成本超出了预算 15%，但其数据采集的信噪比达到了前所未有的水平，总体而言是值得的。分类：支持

示例 2：评论：新政策看似初衷很好，但执行环节的繁文缛节反而增加了基层工作量。分类：反对

示例 3：评论：目前还没有看到明显的改变，再观察一段时间吧。分类：中立

现在，请对以下评论进行分类：

评论：这项政策极大地减轻了我的经济负担，流程也简化了，非常感谢！”

通过这种方式，大模型不仅知道要输出哪个类别，还能从示例中学习“支持”“反对”“中立”三类之间的隐含边界。对于开放式问卷、访谈记录和评论数据，这种少样本提示往往比零样本提示更稳定，也更符合科研人员的具体编码标准。

提示词直接推理不仅可以用于分类，也可以用于信息提取。例如，在文献阅读中，研究人员可以要求大模型从论文摘要中提取“研究对象、研究方法、主要发现和结论”；在实验记录中，可以要求大模型提取“材料名称、实验条件、温度、时间和结果指标”；在访谈分析中，可以要求大模型识别“受访者关注的问题、表达的情绪和提出的建议”。这些任务的共同特点是：原始数据通常是自然语言文本，传统程序很难通过简单规则准确处理，而大模型可以利用上下文语义进行判断。

但是，提示词直接推理并不等同于严格意义上的模型训练，也不能被简单视为“自动标

注真值”。它的输出可能受到多种因素影响。第一，提示词措辞会影响结果。如果分类标准写得模糊，模型可能按照自己的理解进行判断。第二，示例选择会影响结果。如果少样本示例不具有代表性，模型可能学到偏差。第三，模型版本会影响结果。同一提示词在不同模型或不同版本上运行，可能得到不同结果。第四，上下文顺序也可能影响输出，尤其是在长文本和批量样本处理中，模型可能受到前面样本的干扰。

在实际应用中，提示词直接推理适合作为轻量级语义判断工具，而不是替代全部科研判断。它非常适合完成初步筛选、辅助编码、批量预分类和信息整理。例如，研究人员可以先用大模型对数千条评论进行初步分类，再抽取高风险样本进行人工复核；也可以先让大模型提取文献中的研究方法和结论，再由研究人员确认是否符合原文。通过这种方式，大模型承担“第一轮整理”的工作，而研究人员负责“标准制定”和“结果把关”。

5.2.3 智能体算法实验

提示词直接推理能够让大模型快速完成分类、提取和判断任务，但它本质上仍然是让大模型“直接给出答案”。在一些科研场景中，研究人员希望完成的不只是单条文本判断，而是基于已有数据运行一个完整的小型实验：读取训练集和测试集，选择合适的模型，完成模型训练，输出测试结果，并将预测标签保存为文件。此时，就可以借助 **OpenCode**、**Claude Code**、**Cursor** 等编程智能体完成更完整的算法实验流程。

所谓智能体算法实验，是指研究人员不再逐行编写训练代码，而是用自然语言向编程智能体说明任务目标、输入数据和输出要求。智能体会根据任务自动规划实现路径，生成代码，运行程序，查看报错并修复问题，最后输出实验结果和文件。它与提示词直接推理的区别在于：提示词直接推理是让大模型直接判断文本类别；智能体算法实验则是让智能体帮助研究人员搭建一个可运行、可检查、可复查的实验流程。

本节以一个真实的中文商品评论的分类实验为例。实验目录中包含两个数据文件：**train.csv** 和 **test.csv**，其中，**train.csv** 用于训练模型，**test.csv** 用于测试模型性能。每一条样本包含一个评论和对应的标签（“好评”或者“差评”）。研究人员并不需要指定复杂算法，也不需要提前写出完整代码，只需要向 **OpenCode** 输入如指令：

“使用目录下的 **train** 数据集训练一个文本分类模型，再用 **test** 数据集上给出训练模型的性能。分类模型请选择一个适合该任务的模型。测试集的预测标签输出到 **Excel** 文件中，结果输出到文本文件中。”

这段提示词非常简洁，但已经包含了智能体完成任务所需的关键信息。第一，它说明了输入数据，即目录下的 `train` 数据集和 `test` 数据集；第二，它说明了任务目标，即训练一个文本分类模型，并在测试集上评估性能；第三，它把模型选择交给智能体自行决定；第四，它明确了输出要求，即将预测标签保存到 `Excel` 文件中，并将实验结果保存到文本文件中。

智能体执行文本分类实验的基本过程如图 5-6 所示。接收到任务后，`OpenCode` 自动生成了实验代码 `train_eval.py`。该代码读取 `train.csv` 和 `test.csv`，识别其中的 `text` 文本列和 `label` 标签列；随后使用 `jieba` 对中文文本进行分词，并选择 `TF-IDF+Logistic Regression` 作为文本分类模型；接着在训练集上训练模型，在测试集上进行预测；最后计算模型性能，并分别保存 `results.txt` 和 `predictions.xlsx`。由此可见，智能体并不是简单回答“模型效果如何”，而是完成了从数据读取、代码生成、模型训练、结果评估到文件保存的完整闭环。

从实验结果可以看到，本次实验共使用 9600 条训练样本和 7766 条测试样本。模型在测试集上的 `Accuracy` 为 0.8477，`F1-score` 为 0.8797。对于非人工智能专业的研究新生而言，可以暂时把 `Accuracy` 理解为“整体判断正确的比例”，把 `F1-score` 理解为“综合考虑正确率和覆盖率后的整体表现”。这两个数值说明，该基础模型已经能够较好地完成文本分类任务。

进一步查看分类报告和混淆矩阵，可以发现模型对不同类别的表现并不完全一致。例如，模型对“好评”类别的判断相对更可靠，而“差评”类别仍存在一定误判。混淆矩阵还显示，部分“好评”样本被预测成“差评”。这个结果提醒研究人员：模型虽然整体表现不错，但仍需要继续检查误判样本，分析错误是来自文本表达模糊、类别边界不清，还是模型本身能力有限。

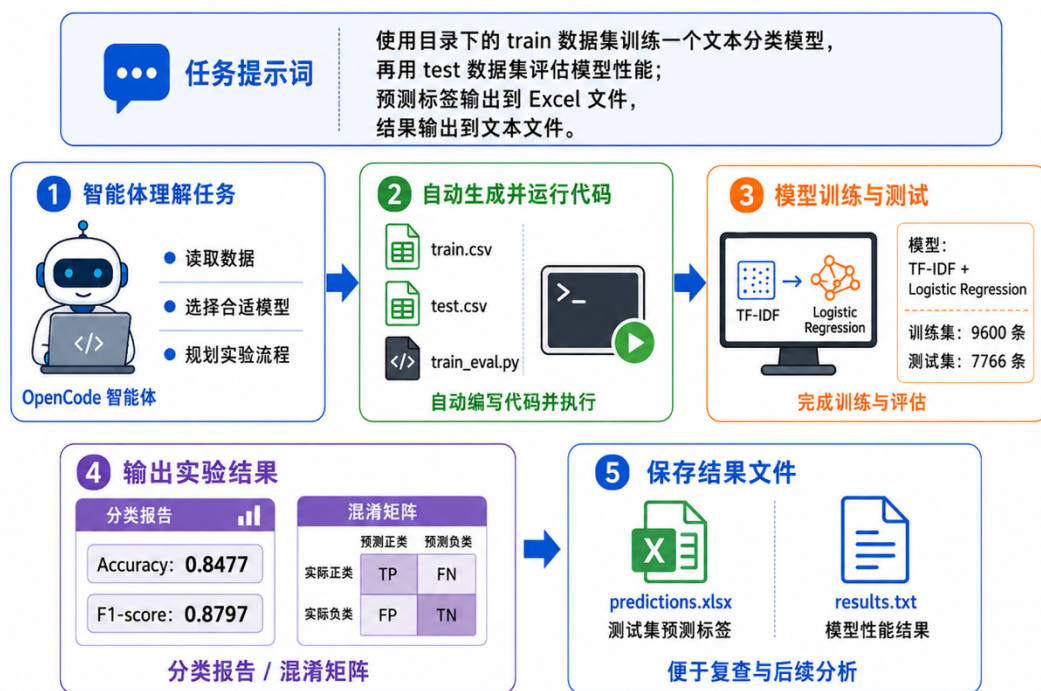


图 5-6 智能体执行文本分类实验的基本过程

本实验还按照提示词要求生成了两个结果文件。`predictions.xlsx` 保存了测试集样本对应的预测标签，方便研究人员打开 Excel 进行筛选、查看和人工复核；`results.txt` 保存了模型名称、样本数量、性能指标、分类报告和混淆矩阵，方便后续写入实验记录或教材示例。对于研究新生来说，这一点非常重要：智能体生成的结果不应只停留在对话框里，而应保存为可以复查、可以共享、可以继续分析的文件。

不过，智能体能够自动输出实验结果，并不意味着这些结果可以不加检查地直接写入论文。研究人员仍然需要进行基本审查。首先，要确认智能体是否正确读取了训练集和测试集，是否把文本列和标签列识别正确。其次，要查看模型是否明显偏向某一类别，例如是否大量把样本预测成同一个标签。再次，要检查图表、分类报告和文字解释是否一致。最后，还要保存代码、结果文件和预测文件，以便后续复查或由其他人重复实验。

因此，智能体算法实验的正确使用方式，不是“把数据交给人工智能后直接相信结果”，而是把智能体视为一位高效的算法助理。它可以帮助研究人员完成重复性强、技术细节多的工作，例如代码编写、模型训练、结果计算和文件保存；研究人员则需要负责确认数据是否可靠、任务是否定义清楚、结果是否合理，以及这些结果是否能够支撑自己的研究问题。

5.2.4 大模型在非结构化数据处理中的优势

在科研工作中，许多高价值信息并不是一开始就以标准表格的形式存在，而是散落在论

文正文、实验记录、访谈材料、开放式问卷、田野笔记、政策文本和会议纪要等非结构化材料之中。这类数据通常没有统一字段，也没有固定格式，往往包含口语化表达、缩写、错别字、长难句、单位混用和上下文指代。对于研究人员而言，真正耗时的工作常常不是最后的统计分析，而是如何把这些混乱材料整理成可以进一步分析的数据表。

在传统自动化处理方式中，研究人员通常依赖正则表达式、关键词匹配或专门编写的 Python 字符串清洗脚本来提取信息。例如，如果要从实验记录中提取温度，传统代码可能会寻找“数字+°C”这样的固定模式；如果要从访谈记录中识别某类观点，可能会依赖特定关键词进行筛选。这类方法在格式高度统一的数据中非常有效，但面对真实科研材料时往往比较脆弱。只要原始文本换了一种表达方式、使用了中文数字，或者将信息分散在前后两句话中，基于字符匹配的程序就可能漏掉关键信息。

大语言模型的优势在于，它并不只是机械地寻找固定字符，而是能够基于上下文进行语义理解。换言之，它能够在一定程度上理解“这句话说的是什么意思”，而不仅仅是“这句话里出现了哪些字”。这使得大模型在处理非结构化数据时具有较强的灵活性。例如，“150°C”“150 摄氏度”“一百五十度”和“温度控制在 150 左右”在字符形式上并不完全相同，但在人类语义理解中都可能指向相同或相近的实验条件。大模型可以利用上下文判断这些表达之间的关系，并尝试将它们统一为标准化结果。

以材料科学研究中的“催化剂反应条件提取”为例。假设研究人员面对如下文献描述：

“在 150°C 的密闭环境中，我们加入了 5g 催化剂 A，然而在另一组对比实验中，温度被大幅提升到了两百摄氏度，使用的是催化剂 B。”

如果使用传统代码提取温度和催化剂名称，程序可能会准确识别“150°C”，但未必能识别“两百摄氏度”；也可能因为催化剂名称和温度分散在不同短语中，而无法正确建立二者之间的对应关系。而大模型可以根据句子的上下文关系判断：第一组实验使用催化剂 A，对应温度为 150°C；第二组实验使用催化剂 B，对应温度为 200°C。此时，研究人员可以向大模型输入如下指令：

请从下面的实验描述中提取每组实验的催化剂名称和反应温度。请将温度统一转换为阿拉伯数字，并用表格形式输出。若某项信息无法从原文判断，请填写‘未说明’，不要自行补充。

大模型提取的催化剂名称和反应温度的可能结果如表 5-1 所示。这个例子说明，大模型可以帮助研究人员处理格式不统一、表达方式多样的文本数据。它不仅能够识别直接出现的数字，也能理解中文数字、上下文转折和实验组之间的对应关系。对于论文摘要、实验记录、访谈材料和开放式问卷等非结构化数据，大模型都可以承担类似的前置整理工作。

表 5-1 大模型提取的催化剂名称和反应温度的可能结果

催化剂名称	反应温度
A	150
B	200

不过，大模型的语义解析能力并不等于结果一定正确。它可能因为原文表达含糊而做出错误推断，也可能在缺少依据时根据常识补全内容。因此，研究人员不能把大模型提取出的结果直接视为最终可信数据。更合理的做法是把大模型作为“前置整理助手”：先由模型从混乱材料中提取初始结果，再通过结构化输出、证据追溯和人工复核进行确认。

基于大模型的非结构化数据清洗流水线如图 5-7 所示。原始材料首先经过大模型的语义解析，被转化为初步结构化结果；随后通过字段规范、单位统一、证据追溯和人工复核，逐步形成可进入 Excel、SPSS、R 或 Python 等工具继续分析的数据表。需要明确的是，大模型并不是要取代这些严谨的数据分析工具，而是可以充当它们的前置“翻译官”，帮助研究人员把原本难以直接分析的文本材料转化为更容易处理的结构化数据。

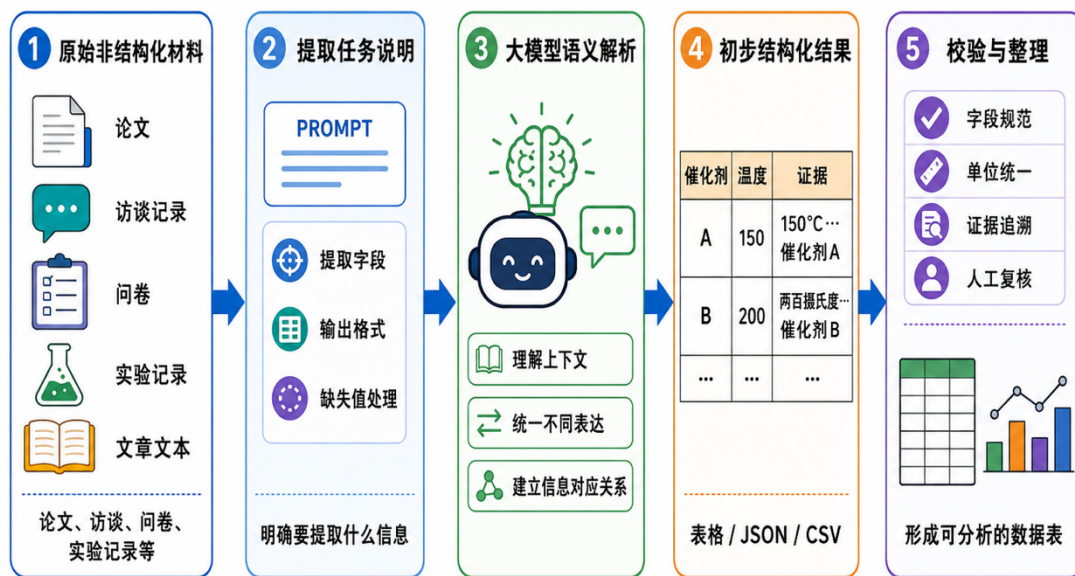


图 5-7 基于大模型的非结构化数据清洗流水线

对于研究新生而言，掌握这一思路非常重要。大模型并不是只会回答问题的聊天工具，它也可以成为科研数据整理流程中的智能助手。但使用它的关键，不是简单地输入“帮我整理数据”，而是清楚说明要提取什么、如何输出、如何处理缺失信息，以及如何保留原文依据。完成非结构化材料的初步整理后，研究人员还需要进一步进入表格清洗、统计计算和图表生成等环节。下一节将讨论大模型和编程智能体如何借助代码执行能力完成自动化数据分析与可视化。

5.3 自动化数据分析与可视化

在前面的内容中，大家已经看到，基于提示词的大模型可以辅助处理非结构化文本，也可以帮助研究人员构建轻量化科研工具。然而，当任务涉及大规模表格数据、统计检验和图表绘制时，仅依靠大模型的语言推理并不可靠。数据分析需要真实的数据读取、数值计算和图表生成过程，因此必须借助代码执行引擎或编程智能体完成可检查的程序计算。代码执行引擎通常允许大模型根据用户需求自动生成 Python 代码，并在相对隔离的运行环境中执行这些代码；编程智能体则可以在本地项目目录中生成脚本、运行程序、读取报错并进行修改。二者的共同点在于：它们把数据分析从单纯的语言回答，推进到“生成代码—运行代码—检查结果”的可复核流程。本节将介绍 AI 如何借助代码执行能力完成表格数据清洗、统计计算、学术图表绘制和结果保存，并说明研究人员在这一过程中应如何检查数据、方法和结论的可靠性。

5.3.1 代码执行引擎：从语言推理到程序计算

代码执行引擎运行的基础载体被称为“沙箱（SandBox）”。对于非计算机专业的研究人员而言，可以将沙箱通俗地理解为系统在云端瞬间为你分配的一台安全、独立且配置完备的虚拟数据工作站。它不仅在物理与逻辑上进行了隔离，防止恶意代码越权操作以保障用户的数据的相对安全，更重要的是，这个云端环境中预先安装了大量科研人员进行科学计算所必需的底层代码库。例如，用于结构化数据清洗与处理的 **Pandas**、用于高级统计推断的 **SciPy**，以及用于绘制学术级图表的 **Matplotlib** 等。在沙箱环境的支持下，大模型处理计算任务的逻辑不再是单向的文字生成，而是演变为一个具有智能体特征的闭环工作流。大模型代码执行引擎的底层代理闭环如图 5-8 所示。

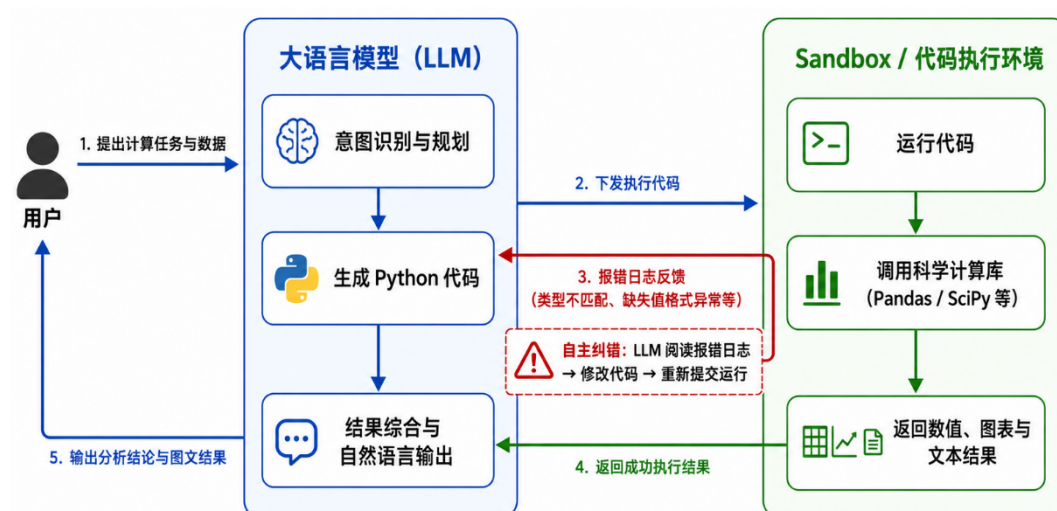


图 5-8 大模型代码执行引擎的底层代理闭环

在这个代理工作流中，第一步是意图识别与规划。模型判断出当前任务需要进行精确的数学计算，便会立刻中止常规的文字推演，转而在后台编写 Python 代码。第二步是代码执行，模型将编写好的代码下发至沙箱环境中运行。第三步是整个流程中最核心的自主纠错机制：如果代码在运行中报错（例如遇到数据类型不匹配、异常格式的缺失值等），沙箱会将详细的报错日志直接反向反馈给语言模型。模型会像真实的程序员一样阅读报错日志，自主寻找逻辑漏洞、修改代码并重新提交运行，如此反复，直至程序成功跑通。最后一步是结果综合，模型提取沙箱计算出的最终结果，结合自然语言输出给用户。

这种底层架构的引入，对科研数据分析的严谨性具有重要意义。以计算一份包含上百万行实验数据的“标准差与极值”为例。如果直接将庞大的数据集粘贴给不支持代码执行的纯语言模型，模型极大可能会因为数据量超出上下文窗口而遗漏信息，或者强行基于前几个数字的规律“猜测”出一个标准差，这个结果在统计学上是完全无效且极其危险的。相反，面对同样的指令，具备代码引擎的模型会立刻在后台生成类似于 `data['value'].std()` 的底层执行代码，并送入沙箱计算。系统将分析过程从单纯语言判断转向可运行、可检查的程序计算。

5.3.2 表格数据的自动化清洗与探索

在科研工作中，CSV 和 Excel 表格是最常见的数据形式。问卷调查结果、实验记录、传感器监测数据、学生成绩、医学测量数据和社会统计数据，最终往往都会被整理成“行—列”结构：每一行代表一个样本，每一列代表一个变量。表格数据看起来比论文、访谈记录等非结构化文本更加整齐，但这并不意味着它已经可以直接进入统计分析。真实数据中常常存在字段命名不统一、缺失值、重复记录、异常值、类别分布不均衡和单位混乱等问题。如果不

先检查这些问题，后续计算结果和图表解释都可能出现偏差。

因此，使用 AI 进行表格数据分析时，不应一开始就要求它“直接画图”或“直接给结论”，而应先让它完成一次基础的“数据体检”。所谓数据体检，是指在正式分析之前，对数据表的基本结构和潜在问题进行检查。研究人员可以上传 CSV 或 Excel 文件，并要求大模型借助代码执行引擎读取表格，检查数据有多少行、多少列，每一列大致属于数值型、类别型还是文本型，哪些字段存在缺失值，是否存在重复样本，数值变量是否有明显异常范围，类别变量是否分布过于集中。

例如，研究人员可以输入如下指令：

“请先对这份表格做数据体检，不要急着画图或下结论。请检查：数据共有多少行和多少列；每一列大致是什么类型；哪些列存在缺失值；是否存在明显异常值；类别变量的分布是否均衡；哪些变量值得进一步分析。请把检查结果整理成简短报告。”

接收到这类任务后，大模型会生成代码读取表格，并调用 Pandas 等工具计算字段类型、缺失比例、数值范围、类别频数和基础统计量。与人工逐列检查相比，这种方式能够快速帮助研究人员了解数据整体状况。表格数据自动化体检的常见检查项如表 5-2 所示。需要注意的是，AI 可以快速发现潜在问题并提出处理建议，但是否删除样本、填补缺失值或修改异常值，仍应由研究人员结合研究背景作出判断。

表 5-2 表格数据自动化体检的常见检查项

检查维度	AI 可以检查什么	研究人员需要确认什么
表格结构	行数、列数、字段名称、数据类型	字段含义是否被正确理解
缺失值	哪些列有空值，缺失比例是多少	缺失是否随机，是否需要删除或填补
异常值	数值是否超出常识范围，如年龄 200 岁、成绩为负数	异常值是录入错误还是重要样本
重复记录	是否存在完全重复或高度相似的样本	是否应删除重复记录
类别分布	类别是否严重不均衡，如某一类占 95%	是否会影响后续统计或建模
单位与格式	单位是否混乱，如 cm 与 m 混用，日期格式不统一	是否需要统一单位和格式
变量关系	初步相关性、分组均值、变化趋势	是否具有学科意义，是否需要进一步检验

在完成基础检查后，AI 还可以辅助进行初步探索。所谓探索，并不是立即得出最终科学结论，而是帮助研究人员发现值得进一步关注的变量关系和数据特征。例如，在问卷数据中，AI 可以比较不同群体的回答差异；在实验记录中，AI 可以计算不同实验组的均值和波动范围；在传感器数据中，AI 可以找出变化幅度较大的时间段；在成绩数据中，AI 可以观察不同科目之间是否存在相关关系。这些结果可以帮助研究人员形成初步判断，决定下一步

应重点分析哪些变量。

不过，AI 自动发现的关系只能被视为“候选线索”，不能直接等同于科学结论。变量之间存在相关性，并不代表一定存在因果关系；某个数值看起来异常，也不一定意味着它应该被删除；某个群体均值较高，也不一定说明该群体具有本质差异。AI 可以帮助研究人员更快地发现可能值得关注的现象，但这些现象背后的原因仍然需要结合研究设计、数据来源和学科知识进行解释。

在自动化清洗方面，也需要保持谨慎。AI 可以提出清洗建议，例如标记缺失值、提示单位不一致、指出重复记录、建议统一类别名称，或者建议对异常值进行复核。但删除样本、填补缺失值、修改异常值和合并类别等关键操作，不应由 AI 在没有说明的情况下自动完成。因为这些操作可能直接改变数据分布，进而影响统计结果和研究结论。比较合理的做法是：先让 AI 列出问题和建议，再由研究人员确认处理方式，最后再让 AI 执行具体修改。

5.3.3 实验数据的统计检验与学术图表生成

探索性数据分析只是科研工作的前奏。对于许多实验研究而言，研究人员最终还需要回答更明确的问题：不同实验组之间是否存在显著差异？某种处理条件是否产生了可观察的效果？实验结果应当如何用图表呈现？这些问题通常涉及统计检验和学术图表生成，不能只依赖模型的语言判断，而应借助代码执行引擎或编程智能体完成真实计算。

在传统科研训练中，研究人员往往需要熟悉 SPSS、Origin、GraphPad、Python 或 R 等工具，并掌握不同统计方法的适用场景。例如，两组数据比较可能使用 t 检验，多组均值比较可能使用方差分析，变量关系分析可能使用相关分析或回归分析。对于研究新生而言，真正困难的地方不仅是点击软件菜单，更在于理解数据结构、选择合适方法、解释统计结果和绘制规范图表。

具备代码执行能力的大模型和编程智能体可以在一定程度上降低这一门槛。前者通常在对话界面中直接接收数据文件，并在沙箱环境中生成和运行 Python 代码；后者则可以在本地或项目目录中读取数据，编写分析脚本，运行程序，并将代码、图表和结果文件保存下来。二者的共同点是：研究人员用自然语言描述实验数据、变量含义和分析目标，系统再调用 Pandas、SciPy、Matplotlib 等工具完成数据读取、统计计算和图表绘制。因此，它们更适合被理解为“统计分析助手”，而不是最终裁判。

下面以“三种肥料对植物株高的影响”为例进行说明。假设研究人员记录了三种不同肥料

处理下植物的最终株高数据：

肥料 A 组（厘米）：12.5，14.1，15.2，13.8，14.0

肥料 B 组（厘米）：18.2，19.5，17.8，20.1，19.0

肥料 C 组（厘米）：13.1，12.0，14.5，13.2，11.8

研究人员关心的问题是：三种肥料处理下的植物株高是否存在显著差异？为了让大模型完成统计检验和图表生成，可以输入如下提示词：

“下面是三种肥料处理下植物株高的数据。请先计算每组的样本量、均值和标准差，再判断三组株高是否存在显著差异。请选择适合该实验设计的基础统计方法进行检验，给出统计量和 p 值，并用通俗语言解释结论。请使用 Python 绘制一张中文图表，展示三组数据的差异，并说明图表表达了什么。最后，请提醒我哪些地方需要人工复核。”

将这段提示词输入具备代码执行能力的大模型后，系统会在后台生成并运行 Python 代码。大模型通常会先整理三组数据，计算每组的均值和标准差；随后根据“三组独立样本均值比较”这一任务，选择单因素方差分析，即 **One-way ANOVA**；接着计算 F 值和 p 值，并生成箱线图、散点图或均值误差条图。大模型进行自动化统计检验与学术绘图的结果展示如图 5-10 所示。

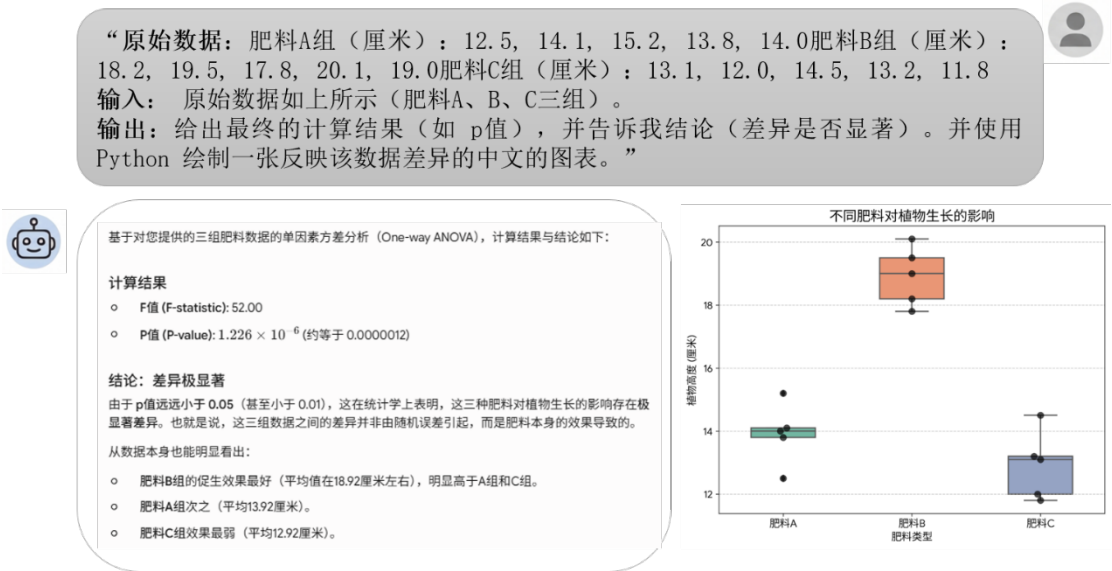


图 5-9 大模型进行自动化统计检验与学术绘图的结果展示

从图 5-9 可以看到，AI 输出的结果不应只关注 p 值，还应包括三个方面。第一，分组描述结果。例如，肥料 A、B、C 三组的平均株高分别是多少，组内波动是否明显。第二，统计检验结果。例如，ANOVA 的 F 值和 p 值是多少，是否达到预设显著性水平。第三，图表解释。例如，箱线图可以展示三组数据的中位数、离散程度和异常值，散点叠加可以显示

每个样本的实际观测值。

如果 p 值小于常用显著性水平 0.05, 可以初步说明三种肥料处理下的植物株高均值并不完全相同。结合本案例中 B 组平均株高明显高于 A 组和 C 组, 可以进一步认为肥料 B 组在该实验数据中表现出更高的促生效果。但这种表述仍需保持谨慎: 统计显著并不等于实际效果一定重要, 也不等于可以直接推断所有植物和所有环境条件下肥料 B 都最好。它只能说明, 在当前实验数据和分析方法下, 三组株高之间存在统计意义上的差异。

图表在这一过程中具有重要作用。相比只报告 F 值和 p 值, 箱线图或散点图能够让研究人员更直观地看到每组数据的分布情况。例如, B 组是否整体高于其他组, A 组和 C 组是否存在重叠, 不同组内部波动是否较大, 是否存在明显异常点。对于科研写作而言, 统计结果和图表应相互支持, 而不是彼此割裂。

不过, AI 自动完成统计检验和图表生成, 并不意味着研究人员可以直接接受所有结论。使用这类工具时, 至少需要复核以下几点: 第一, 数据是否录入正确, 单位是否一致; 第二, AI 是否正确理解了分组变量和测量变量; 第三, 所选统计方法是否符合实验设计, 例如三组是否为独立样本; 第四, 样本量是否过小, 是否需要谨慎解释; 第五, 图表是否真实反映数据, 而没有通过坐标范围、颜色或图形样式夸大差异。

因此, 生成式人工智能在实验数据分析中的价值, 主要体现在降低统计计算和图表生成的技术门槛。它可以帮助研究人员快速完成方法选择、代码运行、结果计算、图表绘制和初步解释, 但不能替代研究人员理解实验背景、检查统计前提和判断科学意义。对于研究新生而言, 合理的使用方式是: 让 AI 帮助跑通分析流程, 再通过代码、图表和结果解释学习统计分析的基本逻辑。

5.3.4 可复现分析

在前面的内容中, 大家已经看到, 具备代码执行能力的大模型或编程智能体可以帮助研究人员完成数据清洗、统计检验和图表生成。然而, 对于科研工作而言, 仅仅在对话框中看到一个结果是不够的。真正可靠的数据分析, 不仅要“看起来正确”, 还应当能够被保存、复查和重复运行。换言之, 可复现比一次性的结论更重要。

所谓可复现分析, 是指研究人员在完成数据分析后, 能够清楚说明: 使用了哪些数据, 执行了哪些代码, 得到了哪些结果, 生成了哪些图表, 以及最终如何解释这些结果。如果只有聊天窗口中的一段文字回答, 而没有保存代码、图表和结果文件, 后续就很难确认计算过

程是否正确，也很难让他人重复同样的分析。

因此，在使用 AI 进行数据分析时，研究人员应尽量要求系统把分析过程保存为一组文件，而不是只输出自然语言结论。一个基本的数据分析输出包通常包括：原始数据文件、分析代码文件、图表文件、结果文件、分析报告和提示词记录。可复现数据分析中建议保存的主要文件如表 5-3 所示。

表 5-3 可复现数据分析应保存的文件

文件类型	示例文件名	作用	需要检查的问题
原始数据	data.csv / raw_data.xlsx	保存分析所依据的数据	是否为正确版本，是否被误删或覆盖
分析代码	analysis.py / notebook.ipynb	记录读取、清洗、统计和绘图过程	是否能重新运行，是否读取了正确字段
图表文件	figure.png / figure.pdf	保存最终可视化结果	图表是否与数据和结论一致
结果文件	results.txt / results.csv	保存统计量、p 值、指标或汇总表	数值是否与代码输出一致
分析报告	report.md / report.docx	说明分析目标、方法、结果和复核点	是否存在过度解释或结论跳跃
提示词记录	prompt.txt	记录向 AI 下达的关键任务说明	是否能说明 AI 实际执行了什么

例如，研究人员可以向大模型或编程智能体输入如下指令：

“请对这份实验数据进行分析，并把所有结果保存下来。请生成 `analysis.py` 保存分析代码，生成 `figure.png` 保存图表，生成 `results.txt` 保存主要计算结果，并生成 `report.md` 用通俗语言说明分析过程、主要结论和需要人工复核的地方。同时，请将本次分析使用的关键提示词保存为 `prompt.txt`。”

这样的提示词能够促使 AI 将一次对话式分析转化为相对规范的分析记录。对于编程智能体而言，还可以进一步要求其在项目目录中创建清晰的文件结构，例如：

```
analysis_project/  
├── data.csv  
├── analysis.py  
├── figure.png  
├── results.txt  
├── report.md  
└── prompt.txt
```

其中，`data.csv` 是原始数据或整理后的数据文件；`analysis.py` 是可重复运行的分析脚本；`figure.png` 是生成的图表；`results.txt` 保存关键计算结果；`report.md` 记录分析目标、方法和结果解释；`prompt.txt` 保存向 AI 下达的关键任务说明。通过这种方式，即使对话结束，研究人员仍然可以重新打开代码、查看图表、核对结果，甚至在修改数据后重新运行整个分析流

程。

这种文件化保存方式也有助于发现错误。例如，如果 `report.md` 中写道“B 组平均值最高”，研究人员可以回到 `results.txt` 检查各组均值是否确实如此；如果图表显示的趋势与文字解释不一致，可以查看 `analysis.py` 中是否选错了变量或分组字段；如果统计结果异常，也可以检查 `data.csv` 是否存在缺失值、异常值或单位错误。也就是说，可复现文件不仅是结果记录，也是错误排查的依据。

存文件并不等于分析一定可靠。研究人员仍然需要检查代码是否读取了正确文件，图表是否对应正确变量，结果文件中的数值是否与图表和报告一致，报告中的解释是否存在过度推断。如果用于正式论文或研究报告，还应保留原始数据版本、分析时间、工具名称和关键参数，以便之后说明分析过程。

5.4 智能体驱动的软件开发流程

前一节讨论的任务通常围绕一次具体的数据分析展开，而在真实科研工作中，研究人员往往还需要进一步构建可反复使用的小型软件工具，例如实验数据自动分析系统、课题组文献问答工具、问卷结果整理平台或图表批量生成程序。与一次性分析不同，科研软件开发更强调功能完整性、运行稳定性和后续可维护性。本节将围绕智能体驱动的软件开发流程展开，说明如何从自然语言需求说明出发，逐步形成最小可用原型，并通过代码生成、运行调试、边界测试和功能迭代，将一个初步工具发展为课题组可复用的科研软件。

5.4.1 从需求说明到最小可用原型

在科研工作中，许多软件工具最初都来自一个具体而朴素的需求。例如，课题组希望自动整理实验数据，希望批量生成统计图表，希望把问卷结果整理成报告，或者希望构建一个可以检索本地文献的小工具。这些想法本身具有明确的科研价值，但它们还不是可以直接使用的软件。要让编程智能体真正完成开发任务，研究人员需要先把这些想法压缩成一个清晰、有限、可运行的最小可用原型。

最小可用原型是指只保留核心功能、能够跑通主要流程、可以被真实试用的第一版工具。它不追求功能完整，也不追求界面精美，而是用于验证一个关键问题：这个工具是否真的能够解决科研中的实际需求。对于科研软件开发而言，第一版原型的目标不是“大而全”，而是“能运行、能演示、能检查、能继续改”。

当功能边界明确后，研究人员就可以把任务交给编程智能体。例如，可以这样描述：

“请帮我开发一个本地运行的实验数据自动分析工具原型系统。第一版只需要支持读取一个 CSV 文件，完成基础数据检查，计算分组均值和标准差，生成一张对比图，并把图表和简短报告保存到 outputs 文件夹。请生成清晰的项目结构、可运行代码、requirements.txt 和 README.md。暂时不要加入用户登录、数据库、云端部署和多人协作功能。验收标准是：使用 sample_data.csv 运行后，能够成功生成 figure.png 和 report.md。”

图 5-10 展示了使用 OpenCode 智能体生成并运行后的实验数据分析系统原型。从界面可以看到，该系统已经具备基本的软件形态：上方提供 CSV 文件选择入口，用户可以通过 Browse 按钮选择数据文件，并点击 Analyze 运行分析；左侧表格展示不同实验组的 N、Mean、Std 和 SEM 等统计结果；右侧自动生成分组对比柱状图；底部显示分析状态，并提供 Save Report 和 Save Chart 按钮，用于保存分析报告和图表。

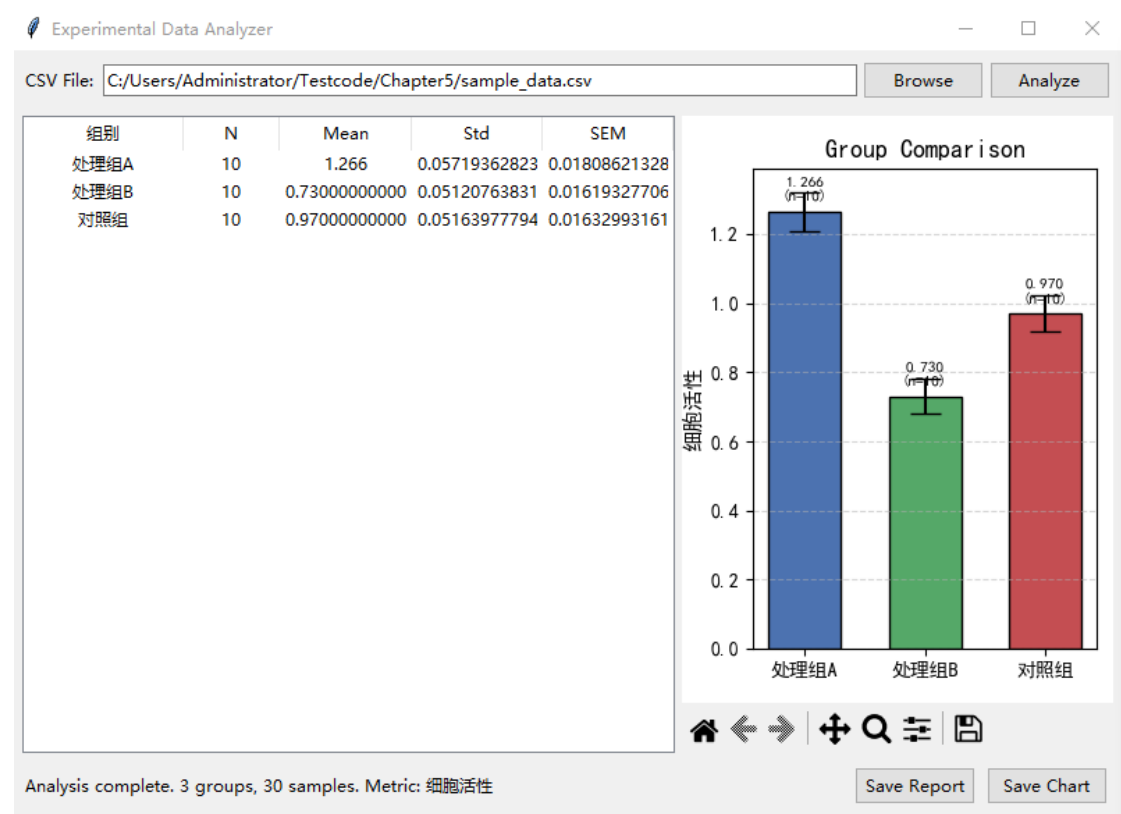


图 5-10 OpenCode 生成的实验数据分析系统最小可用原型

这个系统虽然还不是完整的科研平台，但已经具备最小可用原型的关键特征。第一，它能够接收真实数据文件，而不是只处理写死在代码中的示例数据。第二，它能够完成核心计算，自动输出不同实验组的描述性统计结果。第三，它能够生成可视化图表，帮助研究人员直观比较不同处理组之间的差异。第四，它能够保存结果文件，为后续复查和报告撰写提供基础材料。也就是说，这个原型已经跑通了“选择数据—执行分析—展示结果—保存输出”的

核心流程。

为了避免第一版系统过于复杂，研究人员应主动划定功能边界。最小可用原型的功能边界示例如表 5-4 所示。这里的重点不是限制工具发展，而是让智能体优先完成最关键、最容易验证的部分。

表 5-4 最小可用原型的功能边界示例

内容	第一版原型需要做	第一版暂时不做
数据输入	支持读取一个 CSV 文件	支持所有格式、批量上传
数据分析	计算基础统计量并生成图表	自动选择所有复杂统计模型
结果输出	保存图表和简短报告	生成完整论文级报告
用户界面	提供简单本地界面或命令行入口	精美前端、多用户界面
数据存储	保存到本地文件夹	数据库、云端同步
系统运行	本地运行并提供说明	云端部署、账号权限管理

因此，研究人员向智能体提出任务时，不应简单地说“帮我做一个完整的数据分析系统”，而应明确要求先生成最小可用原型。这个提示词的关键不在于语言复杂，而在于它明确了第一版原型的功能范围。它既说明了系统需要完成的核心流程，也排除了第一版暂时不需要的复杂功能。对于编程智能体来说，这种边界清楚的任务更容易转化为可运行项目；对于研究人员来说，也更容易判断生成结果是否符合预期。

最小可用原型生成后，研究人员首先应检查它是否能够完成核心流程：程序能否启动，数据文件能否选择，分析按钮是否有效，统计结果是否正确显示，图表是否能够生成，报告和图表是否能够保存。如果这些基本条件尚未满足，就不应急于增加新功能。只有当原型能够稳定运行后，才适合进入下一阶段的代码调试、错误修复、边界测试和功能迭代。

5.4.2 智能体的代码生成、运行与自动修复

当最小可用原型的功能边界确定之后，下一步就是让编程智能体进入实际开发阶段。与普通大模型只在对话框中给出代码不同，编程智能体通常可以在项目目录中创建文件、编写代码、安装依赖、运行程序，并根据报错信息进行修改。也就是说，它不只是“生成代码”，而是在一定程度上参与了“生成—运行—检查—修复”的开发闭环。

智能体开发的关键不在于一次性生成完全正确的代码，而在于能够在运行过程中发现问题，并根据报错信息不断修复。其基本过程如图 5-11 所示：研究人员首先提供需求说明和原型边界，智能体生成项目文件并运行程序；如果运行失败，系统读取报错日志，修改代码、依赖或路径后再次运行；如果运行成功，则输出一个可运行的原型工具。

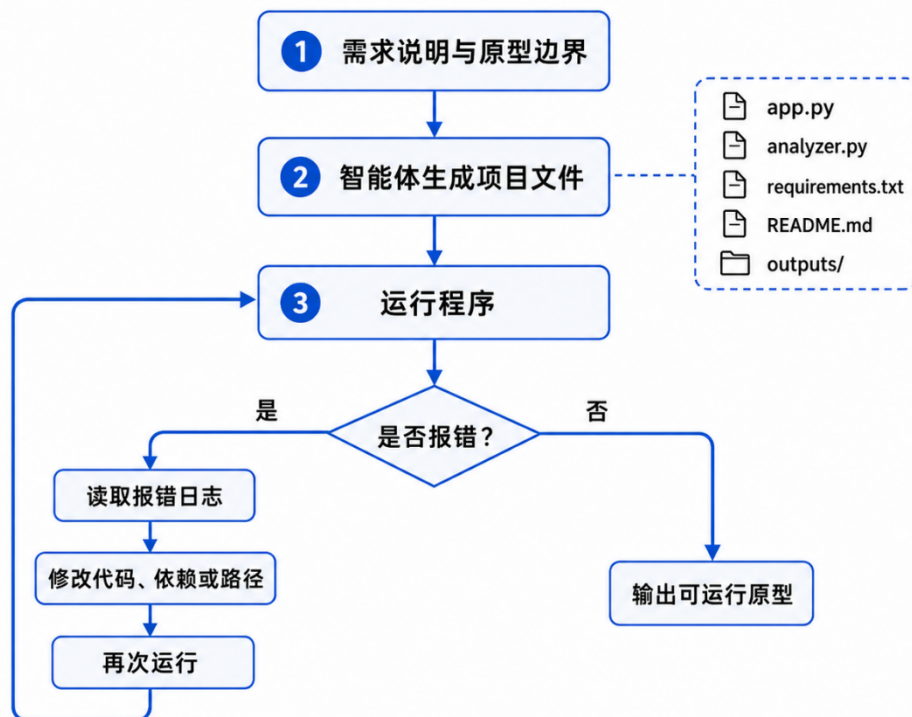


图 5-11 智能体开发中的代码生成与自动修复闭环

以实验数据自动分析工具为例，研究人员向智能体提出任务后，智能体通常会先规划项目结构，例如创建 `app.py`、`analyzer.py`、`requirements.txt`、`README.md` 和 `outputs` 文件夹。随后，它会根据功能要求编写代码：读取 CSV 文件，检查缺失值和字段类型，计算基础统计量，生成图表，并把结果保存为报告文件。这个过程体现了智能体开发与一次性代码生成的区别：它更像是在搭建一个可以运行的小项目，而不是只输出一段孤立代码。

代码生成完成后，真正关键的一步是运行。智能体会在终端中执行程序，例如运行 `python app.py` 或 `python analyzer.py`，检查程序是否能够正常启动，是否能够读取示例数据，是否能够生成 `figure.png` 和 `report.md` 等输出文件。如果程序运行成功，说明原型的核心流程已经初步跑通；如果程序运行失败，智能体则会读取错误信息，并尝试定位问题。

常见错误包括依赖库未安装、文件路径不正确、字段名称与数据表不一致、中文字体无法显示、输出文件夹不存在，或者图表保存失败等。对于这类问题，智能体可以根据终端返回的报错信息自动修改代码。例如，如果程序提示找不到 `outputs` 文件夹，智能体可以在代码中加入自动创建文件夹的语句；如果提示缺少某个 Python 库，智能体可以更新 `requirements.txt`；如果 CSV 文件中的列名不是 `group` 和 `value`，智能体可以改为自动识别列名或在 `README` 中说明所需格式。

这种自动修复能力是编程智能体的重要优势。过去，初学者在运行代码时常常被报错卡住，不知道错误来自环境、依赖、路径还是代码逻辑。智能体可以帮助研究人员完成第一轮

排错，使原型更快进入可运行状态。不过，自动修复并不意味着研究人员可以完全不检查。智能体有时可能为了让程序“跑起来”而采取过于简单的处理方式，例如跳过出错数据、默认填补缺失值，或者忽略某些异常输入。因此，研究人员仍然需要查看修复说明，确认修改是否符合研究目的。

在使用智能体开发科研软件时，可以要求它保留开发过程中的关键信息。例如，每次修复了什么错误，修改了哪个文件，为什么这样修改，当前程序如何运行，以及输出结果保存在哪里。这样做的好处是，研究人员不仅能得到一个可以运行的原型，还能了解系统是如何逐步形成的，为后续维护和扩展提供依据。

智能体的运行结果仍需人工验收。研究人员至少应检查四个方面：第一，程序是否按照预期读取了正确的数据文件；第二，输出文件是否真实生成，而不是只在对话中声称生成；第三，图表和报告是否与输入数据一致；第四，**README** 中的运行说明是否能够被他人照着执行。只有这些条件满足，才能认为原型完成了从“代码生成”到“可运行工具”的转变。

5.4.3 边界用例测试与功能迭代

当智能体生成的科研软件原型能够正常运行后，并不意味着这个工具已经可靠。程序在示例数据上跑通，只能说明核心流程初步成立；在真实使用中，用户可能上传不同格式的数据，填写错误参数，重复运行程序，修改输出路径，或者在缺少依赖的环境中启动系统。如果不经测试，原型很容易在实际使用时出现错误。因此，智能体开发的下一步，是围绕系统使用过程进行测试，并根据测试结果逐步迭代。

这里的测试用例不应只理解为某一类数据样本，而应理解为用于检验系统功能的一组使用场景。以实验数据自动分析工具为例，测试对象不仅包括输入数据是否规范，还包括系统能否正常启动、文件能否上传、分析按钮是否有效、结果是否保存、错误提示是否清楚、重复运行是否覆盖旧文件，以及 **README** 中的运行说明是否可执行。换言之，测试用例要覆盖“用户如何使用这个工具”的完整过程。

系统测试通常可以分为几类。第一类是正常用例，用于确认核心功能是否顺利运行。例如，上传一个格式正确的 **CSV** 文件，检查系统是否能够完成数据读取、统计分析、图表生成和报告保存。第二类是边界用例，用于检查极端但可能出现的情况。例如，数据行数很少、某个分组只有一个样本、数值范围特别大、文件名包含中文或空格。第三类是异常用例，用于检查系统遇到错误输入时是否能够给出清楚提示。例如，上传空文件、缺少必要字段、数

值列混入文字、上传不支持的文件格式，或者输出目录不存在。第四类是操作用例，用于检查用户使用流程是否顺畅。例如，连续多次运行程序、重新上传文件、修改参数后再次分析、关闭程序后重新启动。科研软件原型测试中常见的用例类型如表 5-5 所示。

表 5-5 科研软件原型的常见测试用例类型

测试类型	主要目的	示例
正常用例	检查核心流程能否跑通	上传标准 CSV，成功生成图表和报告
边界用例	检查极端情况是否可处理	样本量很少、数值范围很大、文件名含中文
异常用例	检查错误输入时是否有提示	空文件、缺少字段、数值列混入文字
操作用例	检查用户流程是否顺畅	重复运行、重新上传、修改参数后再次分析
输出用例	检查结果文件是否正确保存	figure.png、report.md 是否生成且内容正确
环境用例	检查不同运行环境下是否可用	缺少依赖库、路径变化、重新安装后运行

研究人员可以要求智能体主动生成这些测试用例。例如：

“请为这个实验数据自动分析工具设计系统测试用例。测试不要只局限于一种数据文件，请覆盖正常使用、边界情况、异常输入、重复运行、输出文件保存和运行环境等方面。请生成测试数据，运行测试，并说明哪些测试通过、哪些测试失败，以及需要修改哪些地方。”

通过这样的测试，智能体可以帮助研究人员发现原型中隐藏的问题。如果程序遇到缺少列名就崩溃，可以要求智能体增加字段检查；如果用户上传不支持的文件格式，可以增加错误提示；如果重复运行导致旧图表被覆盖，可以让智能体在文件名中加入时间戳；如果 README 中的运行命令不完整，可以要求智能体补充安装依赖和启动步骤。这些修改不一定会增加新功能，但会显著提高工具的稳定性和可用性。

功能迭代应遵循“小步修改”的原则。每次只解决一个明确问题，而不是同时加入大量新功能。例如，第一轮迭代可以只解决“缺少字段时给出提示”；第二轮迭代再解决“支持中文文件名”；第三轮迭代再增加“Excel 文件读取”。这种方式更容易定位问题，也更容易判断每次修改是否有效。如果一次要求智能体同时加入用户登录、批量上传、复杂统计模型和云端部署，系统可能变得难以调试。

在迭代过程中，研究人员应要求智能体记录每次修改的原因和结果。例如，在更新日志中写明“修复了输出目录不存在导致图表保存失败的问题”“增加了对空文件的提示”“支持重复运行时自动生成新文件名”等。这样的记录能够帮助课题组成员理解工具的变化，也便于后续维护。

测试的目标不是让最小原型一次性适应所有情况，而是明确它当前能处理什么、不能处理什么。一个科研软件可以暂时不支持复杂输入，但应当清楚告诉用户当前版本的能力范围。对于最小可用原型来说，稳定地完成有限功能，比勉强支持大量功能但结果不可靠更重要。

5.4.4 从原型工具到课题组可复用系统

经过需求收缩、代码生成、运行修复和边界测试后，一个科研软件原型已经可以完成基本任务。但原型工具能够在开发者电脑上运行，并不意味着它已经适合课题组长期使用。课题组可复用系统不仅要“能跑”，还要让其他成员能够看懂、安装、运行、复查和继续维护。因此，从原型工具到可复用系统，关键在于提高工具的稳定性、可理解性和可维护性。

对于课题组成员而言，一个可复用科研工具不能只是一段临时代码，而应当具有清晰的项目结构、完整的运行说明、可测试的示例数据和明确的输出结果。例如，实验数据分析工具可以保留 `app.py` 作为程序入口，`analyzer.py` 负责数据分析逻辑，`sample_data.csv` 用于测试，`outputs` 文件夹用于保存图表和报告，`README.md` 用于说明安装和运行方式。这样的结构能够降低他人接手、复查和修改的难度。

课题组可复用科研工具应具备的基本条件如表 5-6 所示。这些条件并不要求原型工具变成大型软件，而是帮助研究人员判断：这个工具是否已经从“个人临时脚本”发展为“他人也能使用的小型系统”。

表 5-6 课题组可复用科研工具的基本条件

条件	具体要求	作用
项目结构清晰	入口程序、功能模块、数据样例、输出目录分开存放	便于理解、修改和维护
运行说明完整	README 中说明安装依赖、运行命令和输入格式	便于他人独立运行
示例数据可用	提供 <code>sample_data.csv</code> 或示例文件	便于快速验证系统是否正常
示例输出明确	提供示例图表、报告或结果文件	便于判断运行结果是否正确
依赖环境可复现	提供 <code>requirements.txt</code> 或环境说明	减少因环境差异导致的报错
修改记录清楚	记录修复内容、版本变化和新增功能	便于维护和问题回溯
使用边界明确	说明当前支持和不支持的功能	避免用户误用工具
数据安全可控	说明数据保存位置和敏感数据处理注意事项	降低隐私和数据泄露风险

研究人员可以要求智能体进一步完善原型，使其具备课题组复用条件。例如，可以输入：

“请将当前原型整理为课题组可复用版本。请完善项目结构，补充 `README.md`，加入 `sample_data.csv` 和示例输出，整理 `requirements.txt`，增加 `outputs` 文件夹，记录主要修改内容，并说明当前版本支持的输入格式、输出结果和不支持的功能。”

这样的任务不一定会让系统功能变得更多，但会让工具更容易被他人理解和使用。对于科研软件而言，可复用性并不只来自复杂功能，也来自规范组织、清晰说明和稳定运行。尤其是在多人共同使用的课题组场景中，清楚说明“如何运行”“输入数据应满足什么格式”“结果保存在哪里”“哪些情况暂不支持”，往往比继续增加新功能更重要。

课题组可复用系统仍然不同于正式商业软件或大型平台。它不一定需要复杂的用户权限、

云端部署、数据库管理和高并发能力。对于多数科研场景而言，一个本地可运行、说明清楚、输出稳定、便于修改的小型工具，已经能够显著提高课题组的工作效率。是否继续发展为更完整的平台，应根据真实需求、数据安全要求和维护能力决定。

5.5 本章实训

演练一：零代码构建专属趣味科研应用

（1）实训任务

请开发一个基于网页的“实时提示词 Token 与成本估算看板”。

（2）具体要求

目标： 要求其编写一个程序。

输入界面设计：

界面居中提供一个大型的文本输入框，用于让用户粘贴或输入超长的提示词草稿。

提供一个下拉菜单，包含三种常用模型选项及其计费标准（例如：模型 A 0.015 元/千 Token；模型 B 0.03 元/千 Token；模型 C 0.1 元/千 Token）。

输出与交互逻辑：

实时监听： 当用户在文本框中打字时，下方必须实时统计并显示当前的中文字数与英文字符数。

算法估算： 编写一个粗略的换算公式（例如：假定 1 个汉字约等于 1.2 个 Token，1 个英文单词约等于 1.3 个 Token），实时计算出总 Token 预估量。

动态渲染与警告： 实时显示预估花费。同时，在界面底部放置一条“上下文长度进度条”。如果预估 Token 超过 8000，进度条必须动态变成醒目的红色，并弹出警告文本“接近上下文极限，请精简提示词”。

迭代挑战（模仿视频教学）： 如果第一次生成的界面不够美观，请不要放弃。像视频中强调的那样“修改提示词”，要求大模型为你追加一个“深色模式/浅色模式”切换开关，或者增加一个“一键复制文本”的便捷按钮。

演练二：构建非结构化数据的智能分类器

（1）实训任务

假设你正在协助导师处理某学术期刊后台海量的“同行评审意见”。请开发一个“审稿意见情感倾向与决策分类器”，将非结构化的大段文字转化为结构化数据。

（2）具体要求

准备 3 段不同态度的模拟审稿意见（例如：一段极度赞赏但要求改拼写；一段严厉批评其实验设计；一段态度中立要求补充对比实验）。

编写提示词，要求模型不仅要读懂文本，还要执行严格的分类任务。

输出强制约束：必须要求系统以标准的 JSON 格式或 Markdown 表格输出结果。输出的字段必须严格包含：审稿人编号、核心情感倾向（积极/中立/消极）、最终建议判定（接收/小修/大修/拒稿）、提取的核心修改建议（控制在 20 字以内）。

演练三：基于智能体的科研数据分析工具开发与测试

（1）实训任务

使用大模型或编程智能体，开发一个与你所在学科相关的“科研数据自动分析工具”最小可用原型。该工具应能够读取一份 CSV 或 Excel 数据文件，完成基础数据检查、统计分析、图表生成和结果保存。随后，设计若干边界用例对系统进行测试，并根据测试结果对原型进行一次功能迭代。

本实训的重点不是开发一个功能复杂的完整系统，而是体验从自然语言需求说明到可运行原型，再到测试修复和结果保存的完整零代码开发流程。

（2）实训目标

通过本实训，学生应能够理解：零代码编程并不是简单地让 AI“写一个程序”，而是围绕科研任务完成需求描述、原型生成、代码运行、结果保存、边界测试和功能迭代的完整过程。智能体可以帮助研究人员降低软件开发门槛，但研究人员仍需负责控制功能边界、检查运行结果，并判断工具是否真正服务于科研问题。

（3）具体要求

步骤一：确定科研场景与工具目标

结合自己的学科背景，选择一个小型数据分析场景。例如：

- 1) 教育学：分析不同班级学生成绩差异；
- 2) 生物学：比较不同处理组的植物株高或细胞活性；
- 3) 社会学：统计问卷中不同群体的态度差异；
- 4) 环境科学：分析不同时间段的传感器监测数据；
- 5) 医学或心理学：比较不同实验条件下的测量指标。

请用 100—150 字说明该工具要解决什么问题、使用什么数据、输出什么结果。

步骤二：撰写智能体开发提示词

向大模型或编程智能体输入一段结构清晰的开发提示词，要求其生成一个最小可用原型。

提示词应至少包含以下内容：

- 1) 工具目标：这个工具用于什么科研任务；
- 2) 输入数据：支持读取 CSV 或 Excel 文件；
- 3) 核心功能：完成数据体检、基础统计、图表生成；
- 4) 输出结果：保存图表、结果文件和简短报告；
- 5) 功能边界：第一版不需要用户登录、数据库、云端部署等复杂功能；
- 6) 验收标准：使用示例数据运行后，能够成功生成图表和报告。

可参考提示词：

“请你作为科研软件开发智能体，帮我开发一个本地运行的科研数据自动分析工具原型。第一版只需要支持读取一个 CSV 文件，自动检查字段、缺失值和异常值，计算分组均值和标准差，生成一张对比图，并把图表和简短分析报告保存到 `outputs` 文件夹。请生成可运行代码、`requirements.txt`、`README.md` 和 `sample_data.csv`。暂时不要加入用户登录、数据库、云端部署和多人协作功能。验收标准是：运行示例数据后，能够成功生成 `figure.png` 和 `report.md`。”

步骤三：运行原型并保存结果

根据智能体生成的说明运行系统。检查系统是否能够完成以下任务：

- 1) 成功读取示例数据；
- 2) 显示或输出基础统计结果；
- 3) 生成图表文件；
- 4) 保存分析报告；
- 5) 提供基本运行说明。

最终应保存以下文件：分析代码文件，依赖说明文件，示例数据文件，图表文件，结果或报告文件，运行说明文件。

步骤四：设计系统测试用例

不要只使用正常数据测试系统。请从系统使用过程出发，至少设计 5 个测试用例，覆盖正常使用、边界情况和异常输入。

步骤五：进行一次功能迭代

根据测试结果，选择一个最需要修复的问题，让智能体进行修改。例如：

- 1) 增加缺失字段提示；

- 2) 增加空文件检测;
- 3) 支持中文列名;
- 4) 支持 Excel 文件读取;
- 5) 避免重复运行时覆盖旧图表;
- 6) 在报告中加入数据异常提醒。

修改后, 需要重新运行原来的测试用例, 检查问题是否被修复。请记录本次迭代前后的变化。

步骤六: 提交实训结果

- 1) 最终提交以下材料:
- 2) 工具需求说明;
- 3) 智能体开发提示词;
- 4) 生成的项目结构截图;
- 5) 系统运行截图;
- 6) 至少 5 个测试用例及测试结果;
- 7) 一次功能迭代说明;
- 8) 最终生成的图表和报告文件;
- 9) 200 字左右的反思说明: 智能体在哪些环节提高了效率? 哪些结果仍需要人工复核?